

Object-Oriented Programming (OOP) in Python

1. Introduction

Object-Oriented Programming (OOP) is a programming paradigm based on objects. Objects represent real-world entities and contain **data (attributes)** and **behavior (methods)**.

Key Concepts: - **Class:** Blueprint for creating objects. - **Object:** Instance of a class. - **Attributes:** Variables that belong to an object. - **Methods:** Functions that belong to an object. - **Inheritance:** Mechanism to create a new class from an existing class. - **Encapsulation:** Hiding internal data and providing controlled access.

2. Defining a Class

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def greet(self):
        print(f"Hello, my name is {self.name} and I am {self.age} years old.")

# Creating an object
person1 = Person("Alice", 25)
person1.greet()
```

3. Inheritance

```
class Student(Person): # Student inherits from Person
    def __init__(self, name, age, student_id):
        super().__init__(name, age)
        self.student_id = student_id

    def display_id(self):
        print(f"My student ID is {self.student_id}")

student1 = Student("Bob", 20, "S1234")
student1.greet()      # Inherited from Person
student1.display_id() # Specific to Student
```

4. Encapsulation

```
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner
        self.__balance = balance # private attribute

    def deposit(self, amount):
        self.__balance += amount

    def withdraw(self, amount):
        if amount <= self.__balance:
            self.__balance -= amount
        else:
            print("Insufficient funds")

    def get_balance(self):
        return self.__balance

account = BankAccount("Alice", 1000)
account.deposit(500)
account.withdraw(200)
print(account.get_balance()) # 1300
```

5. Additional OOP Concepts

5.1 Method Overriding

```
class Teacher(Person):
    def greet(self):
        print(f"Hello, I am teacher {self.name}")
```

5.2 Class vs Instance Variables

```
class Dog:
    species = "Canis familiaris" # Class variable

    def __init__(self, name):
        self.name = name # Instance variable
```

5.3 Using @property Decorator

```
class Circle:
    def __init__(self, radius):
        self.__radius = radius

    @property
    def radius(self):
        return self.__radius

    @radius.setter
    def radius(self, value):
        if value > 0:
            self.__radius = value
        else:
            print("Radius must be positive")
```

6. Programming Exercises

1. Create a `Car` class with attributes `make`, `model`, and `year`. Add a method to display car details.
2. Extend the `Car` class to `ElectricCar` with an additional attribute `battery_capacity`. Add a method to display battery info.
3. Create a `Rectangle` class with attributes `length` and `width`. Add methods to calculate area and perimeter.
4. Create a `Circle` class with attribute `radius`. Use encapsulation to protect radius and add getter and setter methods. Add a method to calculate the area.
5. Create a `BankAccount` class. Add methods for deposit, withdraw, and checking balance. Ensure the balance cannot be accessed directly.
6. Create a `Student` class with attributes `name`, `roll_number`, and `marks`. Add methods to display student info and calculate average marks.
7. Implement a `Teacher` class that inherits from `Person`. Override the `greet` method to include subject taught.
8. Create a `LibraryBook` class with attributes `title`, `author`, `available`. Add methods to borrow and return the book. Use encapsulation for `available`.
9. Create a `Shape` class and two subclasses `Triangle` and `Square`. Implement area calculation for both using inheritance.
10. Implement a `CompanyEmployee` class. Include `name`, `salary`, and `position`. Add methods to display info and give a raise. Demonstrate encapsulation for salary.